
TEXT CLASSIFICATION USING CLASSICAL ML: SPARSE FEATURES VS CONTEXTUAL EMBEDDINGS

*

Russel Pinkal Gandhi
Roll No: BA2025041
Statistical Machine Learning
Solo Submission

ABSTRACT

This project investigates the impact of feature representation on classical machine learning model performance for multi-class text classification. We compare two fundamentally different feature extraction paradigms: sparse representations (Unigram Bag-of-Words and Bigram TF-IDF) and dense contextual embeddings (DistilBERT CLS token, DistilBERT mean-pooled, and GloVe). Four classical models — Logistic Regression, Linear SVC, K-Nearest Neighbours, and K-Means clustering — are trained on each feature set and evaluated on the Yahoo Answers Topics dataset (10 classes, 8,000 samples) [13]. Dimensionality reduction via Truncated SVD and PCA is applied to all feature spaces. Results demonstrate that contextual embeddings consistently outperform sparse features, with BERT Mean + Linear SVC achieving the best accuracy of 63.44% and Macro F1 of 62.6%, compared to the best sparse baseline of 46.7% (BoW + Linear SVC). We also include GloVe as a bonus feature representation, which achieves competitive performance of 61.2% accuracy.

Keywords Text Classification · Bag-of-Words · TF-IDF · DistilBERT · GloVe · Logistic Regression · SVM · KNN · KMeans · Dimensionality Reduction

1 Introduction

Natural Language Processing (NLP) has undergone a dramatic transformation over the past decade, evolving from hand-crafted sparse feature representations to dense contextual embeddings powered by large pretrained transformer models [4]. However, despite the dominance of deep learning in state-of-the-art NLP benchmarks, classical machine learning models trained on well-designed features remain relevant, interpretable, and computationally efficient baselines [7].

This project explores a central question in NLP from a statistical learning perspective: *how much does the quality of the input feature representation matter when the classifier is held constant?* Specifically, we compare five feature extraction strategies — Unigram BoW, Bigram TF-IDF, DistilBERT CLS embeddings [10], DistilBERT mean-pooled embeddings, and GloVe embeddings [9] — across four classical ML models on a 10-class text classification task.

The Yahoo Answers Topics dataset [13] was chosen for this study due to its diverse topic coverage across 10 categories, its availability on HuggingFace, and its suitability for multi-class classification experiments. We sample 8,000 examples (800 per class) to ensure balanced, tractable experiments.

Our contributions are:

- A systematic comparison of sparse vs dense feature representations across 4 classical ML models (20 total experiments)
- Dimensionality reduction analysis using Truncated SVD [5] and PCA with explained variance plots

*Course: Statistical Machine Learning | **Russel Pinkal Gandhi. BA2025041.**

- Comprehensive evaluation using accuracy, weighted F1, and macro F1 metrics
- Rich visualizations including confusion matrices, t-SNE plots [11], and EDA analyses
- Bonus inclusion of GloVe embeddings [9] as a non-contextual dense baseline

2 Dataset Description and Exploratory Data Analysis

2.1 Dataset

We use the **Yahoo Answers Topics** dataset [13] available on HuggingFace (`yahoo_answers_topics`). The full dataset contains 1.4 million training examples across 10 topic categories. We perform stratified sampling to select exactly 800 examples per class, yielding a balanced dataset of 8,000 samples total. The dataset is split 80/20 into training (6,400 samples) and test (1,600 samples) sets using stratified splitting to preserve class proportions. All data loading and sampling is implemented using scikit-learn [8].

2.2 Class Distribution

The 10 topic categories are: Society & Culture, Science & Mathematics, Health, Education & Reference, Computers & Internet, Sports, Business & Finance, Entertainment & Music, Family & Relationships, and Politics & Government. As shown in Figure 1, our stratified sampling produces a perfectly balanced dataset with exactly 800 samples per class, eliminating class imbalance as a confounding factor.

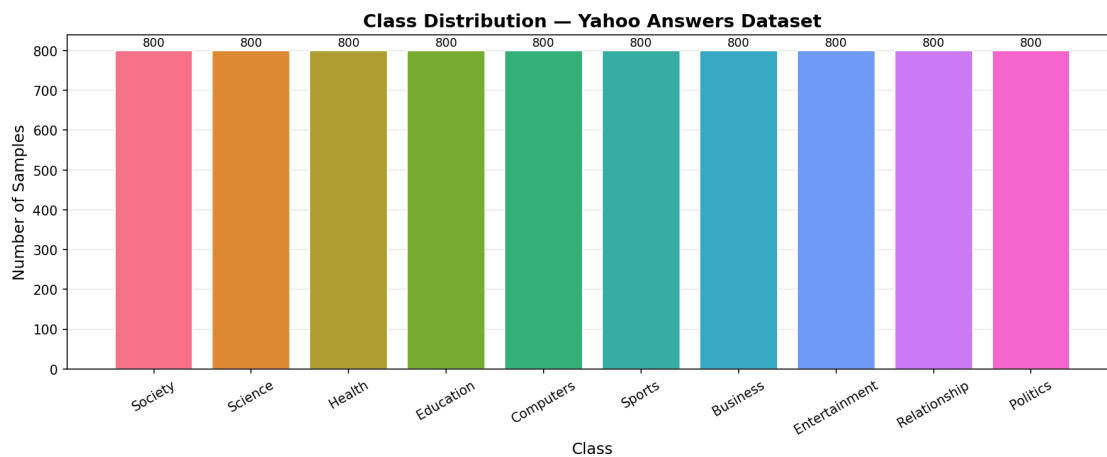


Figure 1: Class distribution across 10 Yahoo Answers topic categories. Perfectly balanced at 800 samples per class due to stratified sampling.

2.3 Text Length Analysis

Figure 2 shows the distribution of text lengths before and after preprocessing. Raw text has a median length of 19 words, while cleaned text (after stop-word removal and tokenization) has a median of 9 words. Both distributions are right-skewed, indicating most questions are short but a few are very long. The significant length reduction after preprocessing confirms effective noise removal.

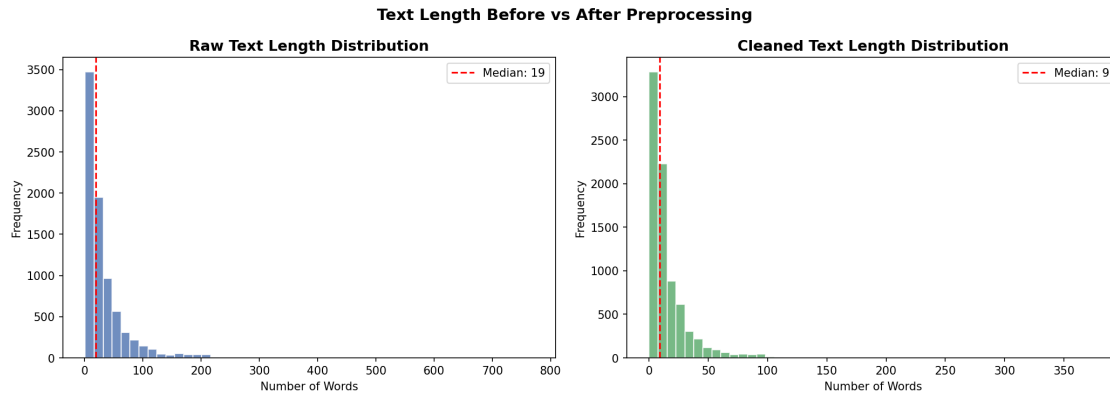


Figure 2: Text length distributions before and after preprocessing. Median drops from 19 to 9 words after stop-word removal.

2.4 Word Clouds

Figure 3 presents word clouds for each of the 10 classes. Each cloud reveals class-specific vocabulary: Sports questions feature words like *team*, *game*, *player*; Politics features *war*, *america*, *bush*; Health features *help*, *doctor*, *feel*. This confirms that the dataset contains discriminative vocabulary per class, making it suitable for text classification.

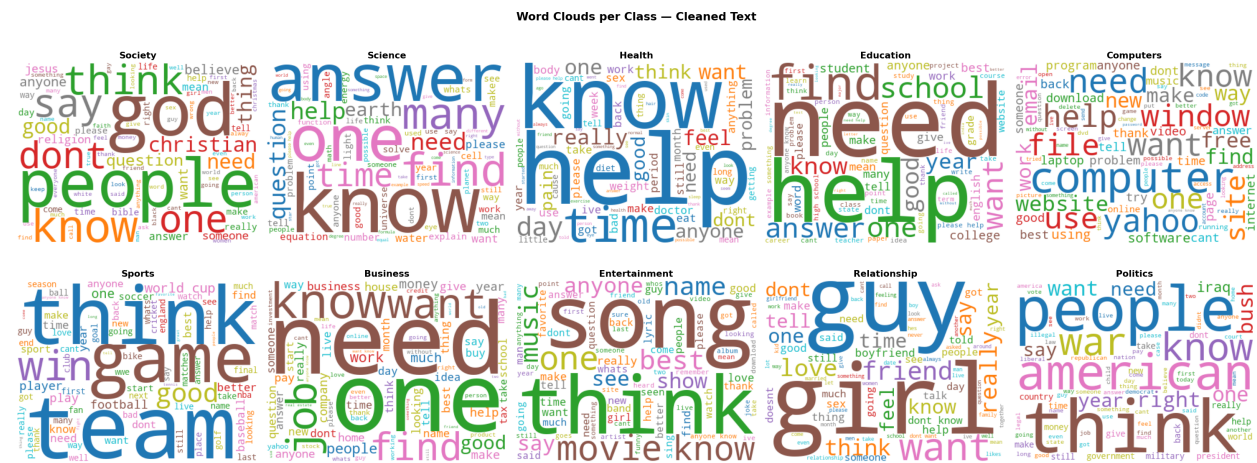


Figure 3: Word clouds for each of the 10 Yahoo Answers topic classes using cleaned text. Word size is proportional to frequency within the class.

3 Methodology

3.1 Text Preprocessing

All text undergoes the following preprocessing pipeline before feature extraction, implemented using NLTK [1]:

1. **Lowercasing:** Convert all text to lowercase
2. **URL removal:** Remove HTTP/HTTPS links using regex
3. **Non-alphabetic removal:** Strip numbers and punctuation
4. **Tokenization:** Smart word tokenization using NLTK's `word_tokenize` [1]
5. **Stop-word removal:** Remove 179 English stop words using NLTK's corpus
6. **Short word removal:** Discard tokens with 2 or fewer characters

3.2 Part A: Sparse Features

Unigram Bag-of-Words (BoW): We use `CountVectorizer` from `scikit-learn` [8] with vocabulary pruning parameters: `max_features=20000` (top 20K words), `min_df=2` (ignore words in fewer than 2 documents), and `max_df=0.95` (ignore words in more than 95% of documents). This yields a sparse matrix of shape (6400, 6638).

Bigram TF-IDF: We use `TfidfVectorizer` [7] with `ngram_range=(2,2)` to capture bigrams (word pairs like “machine learning”). TF-IDF weights rare but informative terms higher than common ones [7]. The same pruning parameters are applied, yielding a matrix of shape (6400, 4638). `sublinear_tf=True` applies log-TF scaling to reduce the impact of very frequent terms.

3.3 Part B: Contextual Embeddings

DistilBERT: We use `distilbert-base-uncased` [10] from HuggingFace Transformers [12] as a frozen feature extractor. No fine-tuning is performed. Sentences are tokenized to a maximum of 128 tokens with padding and truncation. Two embedding types are extracted:

- **CLS token:** The vector at position 0 of the last hidden state, representing the entire sentence [4]. Shape: (n, 768).
- **Mean-pooled:** The average of all token embeddings weighted by the attention mask (excluding padding tokens). Shape: (n, 768).

Embeddings are computed in batches of 64 on a GPU and cached to disk as `.npy` files.

GloVe (Bonus): We use GloVe 6B 300-dimensional vectors [9]. Each sentence is represented as the mean of its constituent word vectors, skipping out-of-vocabulary words. Shape: (n, 300).

3.4 Part C: Models

Four classical ML models are trained on each feature set using `scikit-learn` [8]:

- **Logistic Regression:** Linear classifier with L2 regularization (`max_iter=1000`)
- **Linear SVC:** Support Vector Machine with linear kernel [2] (`max_iter=2000`)
- **KNN:** K-Nearest Neighbours [3] with `k=5` using cosine similarity metric
- **KMeans:** Unsupervised clustering [6] with 10 clusters (`n_init=10`), cluster-to-label mapping via majority voting

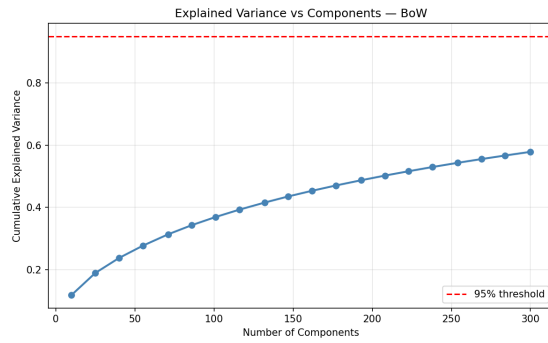
3.5 Part D: Dimensionality Reduction

Truncated SVD [5] is applied to sparse BoW and TF-IDF matrices (reducing to 300 components). Truncated SVD is mathematically equivalent to PCA but operates directly on sparse matrices without dense conversion. **PCA** is applied to dense BERT and GloVe embeddings (reducing to 100 components) [8]. All reducers are fitted on training data only and applied to both train and test sets.

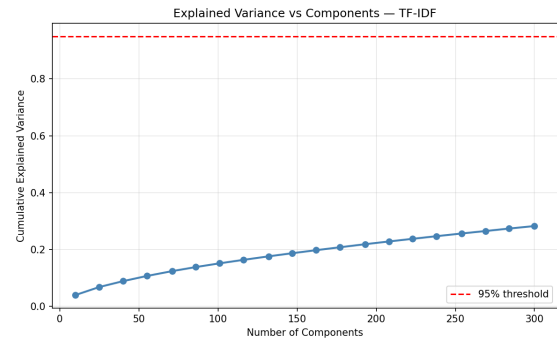
4 Experiments

4.1 Explained Variance Analysis

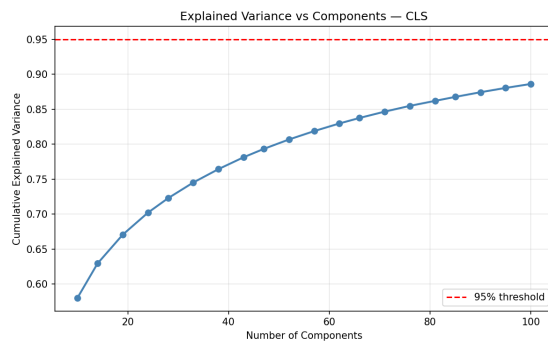
Figure 4 shows cumulative explained variance curves for all five feature types. BoW requires over 300 components to reach 95% variance (only 57.9% at 300 components), and TF-IDF is even more spread out (28.3% at 300 components). In contrast, BERT CLS embeddings [10] achieve 88.6% variance with just 100 PCA components, and mean-pooled embeddings achieve 82.3%. GloVe [9] reaches 78% at 100 components. This demonstrates the information density advantage of contextual embeddings.



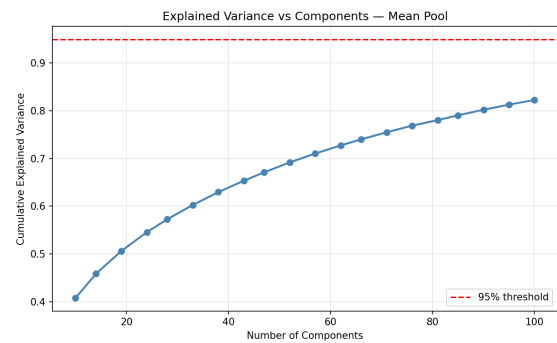
(a) BoW — Truncated SVD



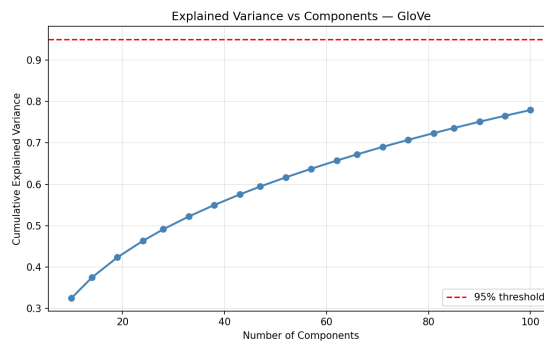
(b) TF-IDF — Truncated SVD



(c) BERT CLS — PCA



(d) BERT Mean Pool — PCA



(e) GloVe — PCA

Figure 4: Cumulative explained variance curves for all 5 feature types. BERT embeddings retain far more information per component than sparse features.

4.2 t-SNE Visualization

Figure 5 shows t-SNE projections [11] of all five feature spaces (2000 samples). BoW and TF-IDF feature spaces show highly mixed, unstructured distributions with no clear cluster separation. BERT CLS and Mean-pooled embeddings show emerging class structure, with some topic clusters becoming visible. This qualitatively confirms why BERT-based features yield superior classification performance.

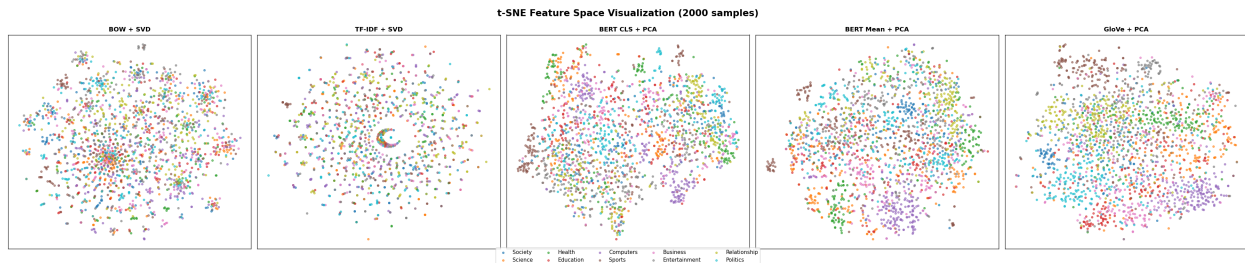


Figure 5: t-SNE visualization [11] of all five feature spaces (2000 samples, 10 classes). BERT embeddings show clearer class separation than sparse features.

4.3 Experimental Setup

All 20 experiments (4 models $\times$ 5 feature sets) are run on the same 80/20 train-test split with `random_state=42` for reproducibility. Training time is recorded for each experiment. KMeans [6] uses majority-vote cluster-to-label mapping evaluated against true labels.

5 Results

5.1 Accuracy and Macro F1

Table 1 summarizes accuracy and Macro F1 scores for all 20 experiments.

Table 1: Accuracy and Macro F1 scores for all models across all feature sets. Best result per metric in **bold**.

Feature Set	Model	Accuracy	Macro F1
BoW + SVD	Logistic Regression	0.4537	0.4560
	Linear SVC	0.4669	0.4620
	KNN	0.2681	0.2700
	KMeans	0.1344	0.0810
TF-IDF + SVD	Logistic Regression	0.2275	0.2210
	Linear SVC	0.2331	0.2270
	KNN	0.2056	0.2040
	KMeans	0.1138	0.0530
BERT CLS + PCA	Logistic Regression	0.6112	0.6050
	Linear SVC	0.6206	0.6120
	KNN	0.5162	0.5150
	KMeans	0.2512	0.2070
BERT Mean + PCA	Logistic Regression	0.6238	0.6190
	Linear SVC	0.6344	0.6260
	KNN	0.5494	0.5450
	KMeans	0.2650	0.2500
GloVe + PCA	Logistic Regression	0.6000	0.5960
	Linear SVC	0.6119	0.6020
	KNN	0.5725	0.5690
	KMeans	0.4025	0.3720

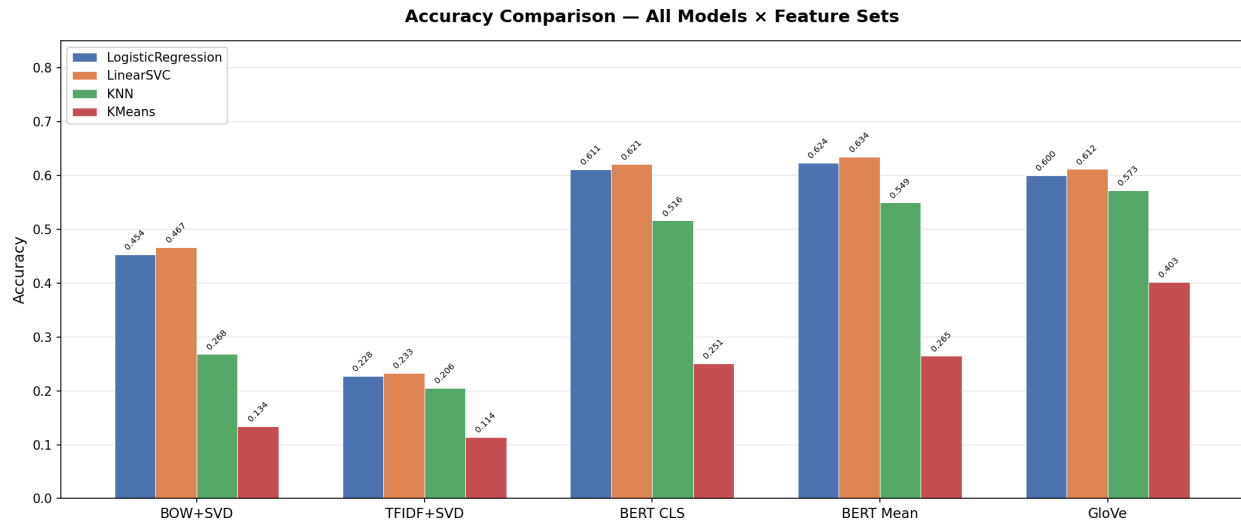


Figure 6: Grouped bar chart comparing accuracy across all models and feature sets.

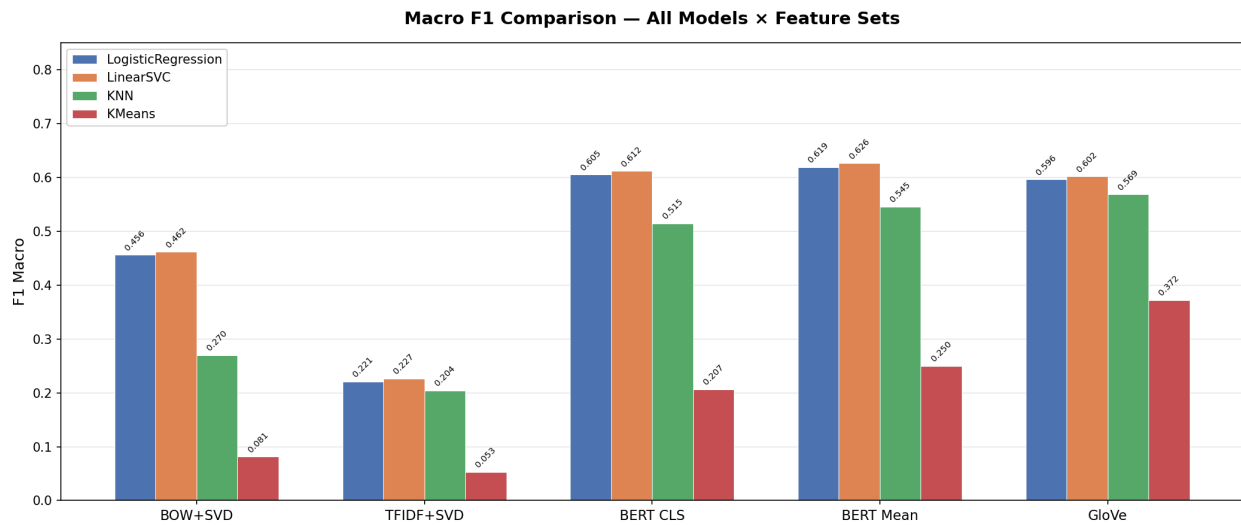


Figure 7: Grouped bar chart comparing Macro F1 across all models and feature sets.

5.2 Confusion Matrices

Figure 8 shows all 20 confusion matrices. BERT-based models exhibit strong diagonal dominance, indicating correct classification. Sparse feature models show more off-diagonal errors. KMeans [6] with sparse features produces nearly random predictions. A common error pattern across all models is confusion between Society & Culture and Politics & Government, which is understandable given their overlapping vocabulary.

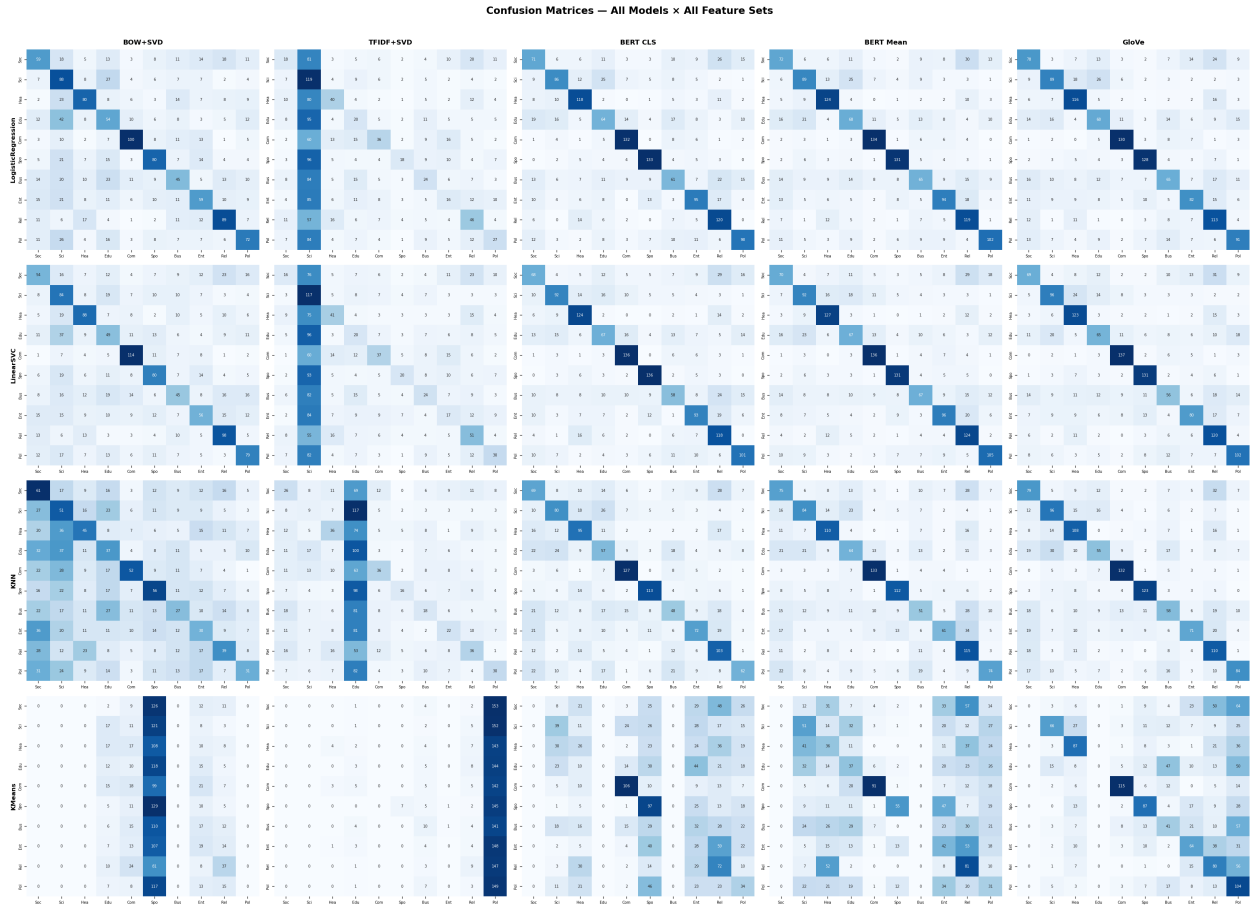


Figure 8: All 20 confusion matrices (4 models $\times$ 5 feature sets). Darker diagonal = better classification.

5.3 Training Time

Figure 9 shows training times on a log scale. KNN [3] has near-zero training time (0.002s) as it stores data without learning. Linear SVC [2] on BoW is the slowest supervised model (7.7s) due to high dimensionality. BERT-based models [10] train faster than BoW-based models despite having more complex features, because PCA reduces them to just 100 dimensions.

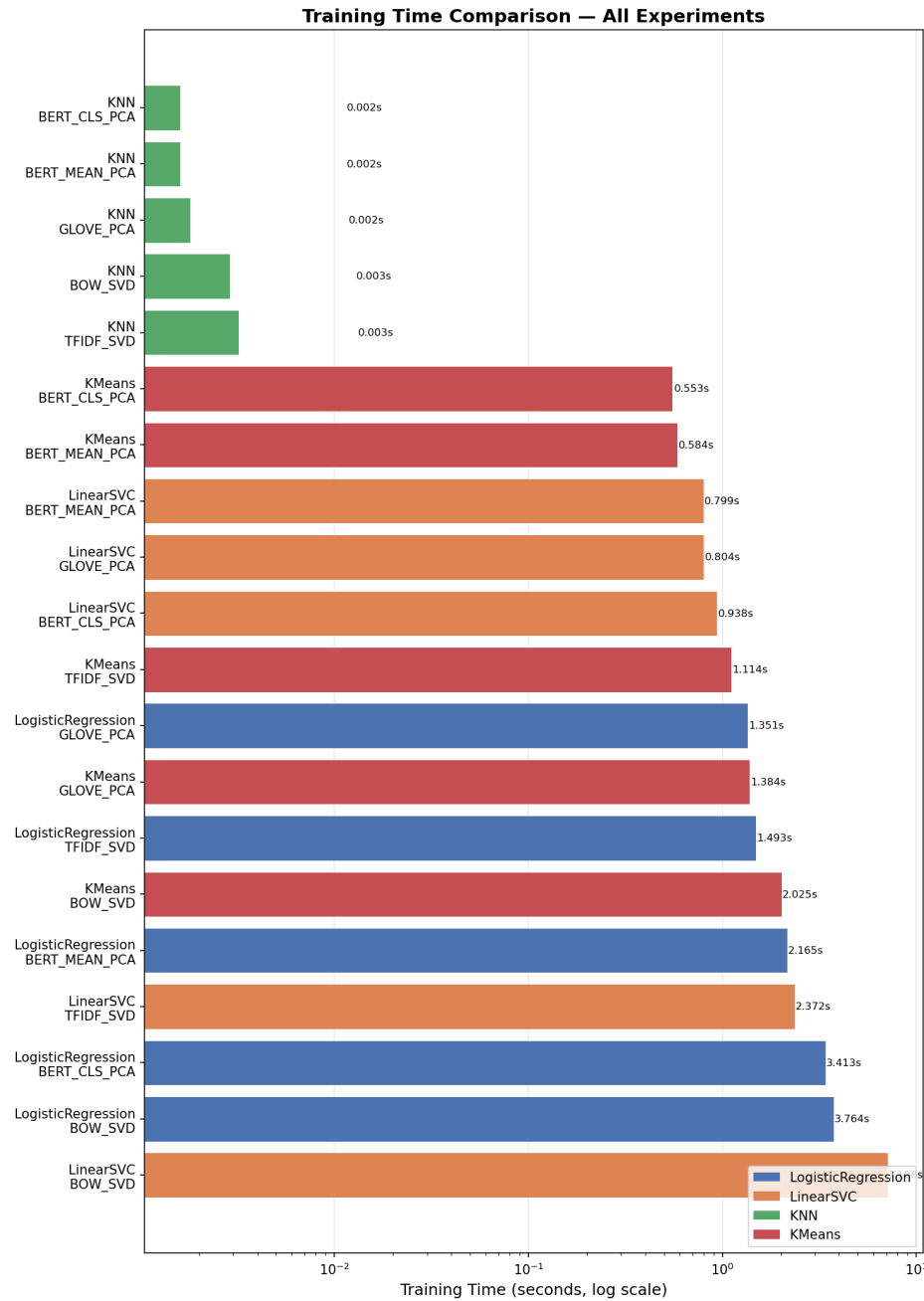


Figure 9: Training time comparison on log scale. KNN is fastest; Linear SVC on BoW is slowest.

6 Discussion

6.1 Sparse vs Dense Features

The most significant finding is the consistent superiority of BERT embeddings [10] over sparse features. BERT Mean + Linear SVC achieves 63.44% accuracy compared to 46.69% for BoW + Linear SVC — a gap of nearly 17 percentage points. This improvement stems from BERT’s ability to capture contextual semantics [4]: the same word receives different representations depending on surrounding context, enabling richer discrimination between topic classes.

6.2 TF-IDF Underperforms BoW

Surprisingly, TF-IDF + SVD consistently underperforms BoW + SVD despite TF-IDF being theoretically superior [7]. This is likely because we used `ngram_range=(2,2)` for TF-IDF (bigrams only), which produces a sparser, less informative vocabulary after pruning compared to unigram BoW. A combined (1,2) ngram range might have produced better results.

6.3 KMeans Performance

KMeans [6] is the weakest model across all feature sets, as expected for an unsupervised method. However, GloVe + KMeans achieves a notably higher accuracy (40.25%) than BERT + KMeans (25–26%), suggesting GloVe’s [9] mean-pooled representations produce more naturally clusterable distributions — possibly because GloVe vectors are designed to be averaged.

6.4 GloVe as a Middle Ground

GloVe [9] achieves competitive performance (61.19% with Linear SVC) despite being a non-contextual embedding. This demonstrates that even static word embeddings capture significant semantic information compared to sparse features, while remaining computationally cheaper than BERT [10].

6.5 Dimensionality Reduction Insight

The explained variance analysis reveals a fundamental difference in the information structure of sparse vs dense features. Sparse features distribute information across thousands of dimensions, making compression lossy [5]. BERT embeddings [10] encode rich information in a compact 768-dimensional space, allowing PCA [8] to retain most information in just 100 components.

7 Conclusion and Learnings

This project demonstrates that feature representation quality has a dominant effect on classical ML model performance for text classification. Moving from sparse BoW/TF-IDF features to contextual BERT embeddings [10, 4] improves accuracy by 15–17 percentage points while also enabling more compact representations.

Key learnings:

- Contextual embeddings (BERT) [10] significantly outperform sparse features even when only used as frozen feature extractors with classical ML classifiers
- Linear SVC [2] is the most robust classifier across all feature types, consistently achieving the best accuracy within each feature set
- KNN [3] benefits more from dense embeddings than from sparse features, improving from 26.8% to 54.9%
- KMeans [6] clustering is fundamentally limited for classification but shows surprisingly good performance on GloVe [9] features
- Dimensionality reduction [5] is essential for sparse features (necessary for tractable training) but mildly harmful for dense features (some information loss)
- The t-SNE [11] visualizations provide intuitive evidence for why BERT features lead to better classification — the feature space is more linearly separable

Future work could explore fine-tuned BERT classifiers [4], RBF-kernel SVMs [2] on BERT features, or ensemble methods combining sparse and dense representations.

Acknowledgments

This project was completed as part of the Statistical Machine Learning course. The Yahoo Answers dataset [13] was sourced from HuggingFace. DistilBERT embeddings [10] were computed using Kaggle GPU resources. GloVe vectors (6B, 300d) [9] were sourced from Stanford NLP.

References

- [1] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. O'Reilly Media, 2009.
- [2] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [3] Thomas Cover and Peter Hart. *Nearest Neighbor Pattern Classification*, volume 13. IEEE, 1967.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics, 2019.
- [5] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. Randomized SVD algorithm used in Truncated SVD implementation.
- [6] Stuart Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.
- [7] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Covers TF-IDF, Bag-of-Words, and classical text representations.
- [8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [9] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics, 2014.
- [10] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [11] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [12] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.
- [13] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. Introduces the Yahoo Answers Topics dataset used in this work.